

小满 RH850U2A16 示例工程说明

目录

1. 基本信息	1
1.1 简介	1
1.2 缩略词	1
1.3 开发环境	1
1.3.1 开发环境获取	2
2. BSW 工程目录结构说明	3
3. 协议栈配置说明	6
3.1 CAN 通信协议栈	6
3.1.1 CAN 通信协议栈概述	6
3.1.2 CAN 通信协议栈配置功能说明	6
3.2 CAN 网络管理协议栈	8
3.2.1 CAN 网络管理协议栈概述	8
3.2.2 CAN 网络管理协议栈配置说明	8
3.2.3 CAN 网络管理协议栈休眠唤醒说明	10
3.3 CAN 诊断协议栈	11
3.3.1 CAN 诊断协议栈概述	11
3.3.2 CAN 诊断协议栈配置说明	11
3.4 存储协议栈	17
3.4.1 存储协议栈概述	17
3.4.2 存储协议栈配置说明	17
3.5 看门狗协议栈	18
3.5.1 看门狗协议栈概述	18
3.5.2 看门狗协议栈配置说明	18
3.6 OS 协议栈	19
3.6.1 OS 协议栈概述	19
3.6.2 OS 协议栈配置说明	19
4. 工程验证	21
4.1 编译及下载	21
4.2 OS 任务	21
4.3 工程运行验证	22

1. 基本信息

1.1 简介

本 Demo 工程集成的 AUTOSAR 协议栈有 CAN 通信、诊断、网络管理、存储、看门狗、OS。各个模块均提供了参考的配置示例，旨在指导用户快速熟悉 AUTOSAR 中各个协议栈的模块的基本配置以及各个模块间的关联关系。

工程名	工程类型
Demo_RH850U2A16_V2504_BSW_ConfigProject	BSW 配置工程
Demo_RH850_U2A16_V2504_GHS20220104_Project	BSW 集成工程

1.2 缩略词

缩略语	中文解释
UDS	统一的诊断服务
CAN	控制器局域网络
NM	网络管理
CanIf	CAN 接口模块
CanSm	CAN 状态管理模块
ComM	通信管理模块
EcuM	ECU 状态管理模块
NvM	非易失性存储管理
FEE	Flash 模拟 Eep
DCM	诊断通信管理
DEM	诊断事件管理
CANTP	CAN 传输层
ComM	通信管理
WDG	看门狗
WdgIf	看门狗接口模块
WdgM	看门狗管理模块
E2E	(End-to-End) 通信安全协议
OS	操作系统

1.3 开发环境

工具描述	名称版本
------	------

编译器&IDE	GreenHills win64_v800_202214
DaVinci 工具	DaVinciConfigurator
MCAL	AUTOSAR_RH850_U2Ax_MCAL_Ver43.03.00_QM_MP
硬件开发板	RH850U2A_292 V2.4_20240422 WP_RH850U2A16292OTPE_V2.0
调试器	Renesas E2

1.3.1 开发环境获取

上述开发环境需用户自行获取。

2. BSW 工程目录结构说明

AUTOSAR 基础软件集成工程的目录结构如下：

Demo_RH850_U2A16_V2504_GHS20220104_Project >		在 Demo_RH850_U2A16_V
   排序 ▾  查看 ▾ ...		
名称		修改日期
 ASW		2025/6/17 20:36
 BSW		2025/5/15 11:42
 LinkFile		2025/6/17 20:36
 Main		2025/6/17 20:36
 StartUp		2025/6/17 20:36
 debug.gpc		2025/6/17 20:36
 Demo_RH850_U2A16_V2504_GHS20220104_Project.ghsmc		2025/6/17 20:36
 Demo_RH850_U2A16_V2504_GHS20220104_Project.gpj		2025/6/17 20:36
 ProjectSetting.gpj		2025/6/19 15:06
 release.gpc		2025/6/17 20:36
 SourceCode.gpj		2025/6/17 20:36

各文件夹释义：

ASW： 存放应用代码，供客户添加应用代码。

BSW： 存放 BSW 相关代码，包括 AUTOSAR 各个协议栈源代码和配置、MCAL 的源代码和配置、复杂驱动的源代码和配置等。

名称	修改日期
 CommonInclude	2025/5/20 17:19
 Communication	2025/5/19 17:36
 Config	2025/5/19 17:36
 Libraries	2025/5/19 17:36
 MCAL	2025/5/19 17:36
 Memory	2025/5/19 17:36
 SystemServices	2025/5/19 17:43

CommonInclude： 存放共用的头文件，比如 Std_Types.h、Compiler.h、ComStack_Types.h 等。

Communication： 存放通信相关的代码，包含 Can、Lin、Ethernet、FlexRay

等。

名称	修改日期
CAN	2025/5/19 17:44
Common	2025/5/19 17:44

Config: 存放 BSW 配置相关的代码，包含 BSW 的配置、MCAL 的配置、CDD 的配置等。

名称	修改日期
BSW_Config	2025/5/20 17:23
MCAL_Config	2025/5/19 17:36

Libraries: 存放 CRC、E2E 等通用库代码。

MCAL: 存放 MCAL 各模块的源代码、CanTrcv、LinTrcv、EthPhy、CDD 模块源代码。

Demo_RH850_U2A16_V2504_GHS20220104_Project > BSW > MCAL > 在 MCAL 中搜索

排序

查看

名称	修改日期
can	2024/5/11 18:18
driver	2024/5/11 18:18
fls	2024/5/11 18:18
general	2025/5/15 11:42
gpt	2024/5/11 18:18
lib	2024/5/11 18:18
mcu	2024/5/11 18:18
port	2024/5/11 18:18
wdg	2024/5/11 18:18

Memory: 存放存储相关模块的源代码。

名称	修改日期
Fee	2025/5/15 11:08
MemIf	2025/5/19 17:36
NvM	2025/5/19 17:36

SystemServices: 存放系统服务相关模块的源代码。

名称	修改日期
 BswM	2025/5/19 17:36
 ComM	2025/5/19 17:36
 Dem	2025/5/19 17:36
 Det	2025/5/19 17:36
 EcuM	2025/5/19 17:36
 Os	2025/5/19 17:36
 WdgIf	2025/5/19 17:43
 WdgM	2025/5/19 17:43

LinkFile: 存放链接文件。

Demo_RH850_U2A16_V2504_GHS20220104_Project > LinkFile		在 LinkFile 中搜索
		↑↓ 排序 ≡ 查看 ...
名称	修改日期	
 App_Common_Sample.ld	2025/6/19 15:02	
 App_U2A16_Sample.ld	2025/6/19 15:02	

3. 协议栈配置说明

3.1 CAN 通信协议栈

3.1.1 CAN 通信协议栈概述

CAN 通信协议栈涉及到的软件模块主要有 CAN、CanIf、PduR、Com、ECUC 模块，其中各个模块的主要功能如下表：

CAN 通信栈各配置模块介绍

模块名	功能
Can	主要配置 CAN 控制器的波特率，CAN 报文的收发邮箱。
CanIf	CanIf 模块主要处理上层模块与底层驱动的之间 PDU 的传递，为上层模块提供统一的接口来管理不同的 CAN 硬件模块。
EcuC	用于辅助配置工具完成配置的模块。主要提供 Pdu 的定义，其它模块通过关联 EcuC 中 Pdu，相互关联起来。
PduR	PDU Router 主要为通讯接口模块（CANIF）、传输协议模块（CAN TP、J1939 TP）、诊断通讯管理模块（DCM、J1939DCM）以及通讯模块（COM、LDCOM）以及 IPDUM、SECOC 等模块提供基于 I-PDU 的路由服务。
Com	COM 模块主要提供 I-PDU 和信号相关管理功能

3.1.2 CAN 通信协议栈配置功能说明

CAN 通信协议栈配置的发送报文说明：

报文名	CANID	说明
CAN0_Tx_0x300_Cyclic	0x300	周期报文，周期时间：500ms 该报文下的信号均配有 UB 位
CAN0_Tx_0x301_Event	0x301	事件报文
CAN0_Tx_0x302_Mixed	0x302	混合报文，正常周期：500ms，触发

		后连发 3 帧，周期为： 50ms
CAN0_Tx_0x303_Cyclic_Counter	0x303	周期报文，周期时间： 500ms，带 RollingCounter
CAN0_Tx_0x350_Cyclic_PN17	0x350	周期报文，周期时间： 100ms 受 PN17 控制
CAN0_Tx_0x351_Cyclic_PN29	0x351	周期报文，周期时间： 100ms 受 PN29 控制
CAN0_Tx_0x360_E2E_P01	0x360	周期报文，周期时间： 100ms E2E 报文，DATAID: 0x1234

CAN 通信协议栈配置的接收报文说明：

报文名	CANID	说明
CAN0_Rx_0x200_Cyclic	0x200	周期报文，周期时间： 500ms 该报文下的信号均配有 UB 位
CAN0_Rx_0x201_Event	0x201	事件报文
CAN0_Rx_0x202_Mixed	0x202	混合报文，正常周期： 500ms，触发后 连发 3 帧，周期为： 50ms，包含信号超 时 2500ms
CAN0_Rx_0x203_Cyclic_Counter	0x203	周期报文，周期时间： 500ms，带 RollingCounter
CAN0_Rx_0x250_Cyclic_PN17	0x250	周期报文，周期时间： 100ms 受 PN17 控制
CAN0_Rx_0x251_Cyclic_PN29	0x251	周期报文，周期时间： 100ms 受 PN29 控制
CAN0_Rx_0x260_E2E_P01	0x260	周期报文，周期时间： 100ms E2E 报文，DATAID: 0x1234

3.2 CAN 网络管理协议栈

3.2.1 CAN 网络管理协议栈概述

CAN 通信协议栈涉及到的软件模块主要有 Can、CanIf、CanSM、EcuC、NM、CanNm、ComM 模块，其中各个模块的主要功能如下表：

网络管理栈各配置模块介绍

模块名	功能
Can	主要配置 CAN 控制器的波特率，CAN 报文的收发邮箱。
CanIf	CanIf 模块主要处理上层模块与底层驱动的之间 PDU 的传递，为上层模块提供统一的接口来管理不同的 CAN 硬件模块。
EcuC	用于辅助配置工具完成配置的模块。主要提供 Pdu 的定义，其它模块通过关联 EcuC 中 Pdu，相互关联起来。
Nm	NmIf 模块主要包含两个功能：NmIf 模块是 ComM 与 CanNm 之间的适配层；网络管理协调功能，协调不同总线 channel 的 ECU 节点实现网络的同步睡眠。
ComM	ComM 模块封装了控制底层的通信服务。通信管理模块从通信请求者那里收集总线通信访问请求，并协调这些请求，主要目的是：为每个 Channel 设置一个状态机控制一个 ECU 的多个通信总线通道。
CanSM	主要功能是与通信硬件抽象层和系统服务层产生交互，为每一个 CAN 通信总线定义一个总线相关的状态管理，并为相关的总线提供流控制。
CanNM	负责实现 ECU 的状态切换。比如合适进入睡眠、是否保持正常的网络状态等。

3.2.2 CAN 网络管理协议栈配置说明

CAN 网络管理的接收报文 ID 范围为 0x500-0x5ff,

CAN 网络管理的发送报文 ID 为 0x501

CanNM 的主要配置参数如下表所示：

配置项	配置参数
-----	------

CanNmGlobalPnSupport	TRUE
CanNmComUserDataSupport	TRUE
CanNmMainFunctionPeriod	0.005
CanNmPassiveModeEnabled	FALSE
CanNmPnEiraCalcEnabled	TRUE
CanNmPnResetTime	2.5S
CanNmActiveWakeupBitEnabled	TRUE
CanNmCarWakeUpRxEnabled	FALSE
CanNmImmediateNmCycleTime	0.02S
CanNmImmediateNmTransmissions	10
CanNmMsgCycleOffset	0.0
CanNmMsgCycleTime	1.0S
CanNmMsgTimeoutTime	0.001S
CanNmNodeId	1
CanNmPduCbvPosition	CANNM_PDU_BYTE_1
CanNmPduNidPosition	CANNM_PDU_BYTE_0
CanNmPnEnabled	TRUE
CanNmPnHandleMultipleNetworkRequests	FALSE
CanNmRepeatMessageTime	3S
CanNmRetryFirstMessageRequest	FALSE
CanNmTimeoutTime	3.0S
CanNmWaitBusSleepTime	1.5S
CanSMBorCounterL1ToL2	10
CanSMBorTimeL1	0.1S
CanSMBorTimeL2	1.0S
CanSMBorTimeTxEnsured	FALSE
CanSMEEnableBusOffDelay	FALSE

3.2.3 CAN 网络管理协议栈休眠唤醒说明

设置唤醒源主要包含两个为远程唤醒和本地唤醒

唤醒源	说明
EcuMWakeupSource_CAN	被动唤醒，需要检测总线上是否为网管报文
EcuMWakeupSource_Local	主动唤醒，用户请求后就会立即请求网络

在开发板上，ECU 一上电主动请求网络，释放网络后 ECU 休眠时直接调用 Mcu_PerformReset 进行复位。

3.3 CAN 诊断协议栈

3.3.1 CAN 诊断协议栈概述

CAN 诊断协议栈涉及到的软件模块主要有 Can、CanIf、CanTP、EcuC、DCM、DEM 模块，其中各个模块的主要功能如下表：

诊断栈各配置模块介绍

模块名	功能
Can	主要配置 CAN 控制器的波特率，CAN 报文的收发邮箱。
CanIf	CanIf 模块主要处理上层模块与底层驱动的之间 PDU 的传递，为上层模块提供统一的接口来管理不同的 CAN 硬件模块。
EcuC	用于辅助配置工具完成配置的模块。主要提供 Pdu 的定义，其它模块通过关联 EcuC 中 Pdu，相互关联起来。
PduR	PDU Router 主要为通讯接口模块(CANIF)、传输协议模块(CAN TP、J1939 TP)、诊断通讯管理模块（DCM、J1939DCM）以及通讯模块（COM、LDCOM）以及 IPDUM、SECOC 等模块提供基于 I-PDU 的路由服务。
CanTp	CANTP 模块实现依据 ISO15765-2 标准规范中定义的 CAN 总线数据在传输层的数据接收发送功能。
Dcm	依据 ISO15765-3 和 ISO14229-1 标准描述，实现诊断请求报文的解析，响应(正响应和负响应)与执行。
Dem	实现诊断故障的存储与管理功能，提供 API 接口供其他模块读取 DTC 和对应的冻结帧数据和扩展数据。

3.3.2 CAN 诊断协议栈配置说明

CAN 诊断协议栈的 CANID 如下表：

CANID 类型	CANID
物理寻址 Physical Request CAN ID	0x708

功能寻址 Functional Request CAN ID	0x7DF
物理响应 Physical Response CAN ID	0x709

Demo 工程中配置的诊断服务有如下表所示：

Service ID 服务标识符	Diagnostic Services Name 诊断服务名称	是否支持 物理寻址	是否支持 功能寻址	是否支持 正确响应抑制	Sub-functions 子功能	Diagnostic Session 诊断模式			Security Level 安全级别			NRC Applies (优先参考ISO 14229)	Comments 备注
						Default(01)	programming DiagnosticS ession(02)	Extended(03)	Locked	Level 1	Level 2		
0x10	DiagnosticSessionControl 诊断模式控制	√	√	√	0x01 DefaultSession 0x01 默认模式	Y	Y	Y	Y	Y	Y	13(最小长度) 12(子服务不支持) 13(长度不匹配) 22(条件不满足)	10 02服务为应用跳转 BOOT功能，为先复位。 BOOT段实现响应
					0x02 ProgrammingSession 0x02 程序设计模式	N	Y	Y	Y	Y	Y		
					0x03 ExtendedDiagnosticSession 0x03 扩展诊断模式	Y	N	Y	Y	Y	Y		
0x11	EcuReset 电控单元复位	√	√	√	0x01 HardReset 0x01 硬件复位	Y	Y	Y	Y	Y	Y	13(最小长度) 12(子服务不支持) 13(长度不匹配) 33(未安全解锁) 22(条件不满足)	0x01 硬件复位 当前用软件复位实现
					0x03 SoftReset 0x03 软件复位	Y	Y	Y	Y	Y	Y		
0x27	SecurityAccess 安全访问	√	x	x	0x03: Security Level1 (Request Seed) 0x03: 安全访问等级1 (请求种子) 0x04: Security Level1 (Send Key) 0x04: 安全访问等级1 (发送密钥) 0x05: Security Level2 (Request Seed) 0x05: 安全访问等级2 (请求种子) 0x06: Security Level2 (Send Key) 0x06: 安全访问等级2 (发送密钥)	N	N	Y	Y	Y	Y	7F 13 12 22 24 35 36 37	
0x28	CommunicationControl 通信控制	√	√	√	0x00 EnableRxAndTx 0x00 使能收发	N	N	Y	Y	Y	Y	13(最小长度) 28 xx 02网络管理通信 参数 13(长度不匹配) 31(参数错误) 22(条件不满足)	28 xx 01应用通信参数 28 xx 02网络管理通信 参数 28 xx 03应用通信和网络 管理通信参数
					0x01 EnableRxAndDisableTx 0x01 使能收发禁止发	N	N	Y	Y	Y	Y		
					0x02 DisableRxAndEnableTx 0x02 禁止收发使能发	N	N	Y	Y	Y	Y		
					0x03 DisableRxAndTx 0x03 禁止收发	N	N	Y	Y	Y	Y		
0x3E	TesterPresent 诊断设备在线	√	√	√	0x00 ZeroSubFunction 0x00 Zero 子功能	Y	Y	Y	Y	Y	Y	13(最小长度) 12(子服务不支持) 13(长度不匹配)	
0x85	ControlDTCSetting 控制DTC设置	√	√	√	0x01 ON 0x01 打开DTC设置	N	N	Y	Y	Y	Y	13(最小长度) 12(子服务不支持) 13(长度不匹配) 31(参数错误) 22(条件不满足)	
					0x02 OFF 0x02 关闭DTC设置	N	N	Y	Y	Y	Y		
0x22	ReadDataByIdentifier 读取数据 (通过标识)	√	√	N/A	N/A -> 22服务无子服务	参照DIDList sheet页						13(最小长度) 13(最大长度) 33(未安全解锁) 22(条件不满足) 31(参数错误)	一次最多读5个DID
0x2E	WriteDataByIdentifier 写入数据 (通过标识)	√	x	N/A	N/A -> 2E服务无子服务	参照DIDList sheet页						33(未安全解锁) 13(最小长度) 31(DID不支持) 13(长度不匹配) 31(参数错误&会话不支持) 22(条件不满足) 72(写出错)	
0x2F	InputOutput (输入输出控制单元)	√	x	N/A	N/A -> 2F服务无子服务	N	N	Y	N	Y	Y	33(未安全解锁) 13(最小长度) 31(RID不支持) 13(检测品长度) 31(控制状态或MASK) 22(条件不满足)	
0x31	RoutineControl 例程控制	√	x	x	0x01 StartRoutine 0x01 开始执行	N	Y	Y	Y	Y	Y	13(最小长度) 31(RID不支持) 33(未安全解锁) 12(子服务不支持) 13(长度不匹配) 31(参数错误) 22(条件不满足) 24(请求顺序错误)	
					0x02 StopRoutine 0x02 停止执行	N	N	N	N	N	N		
					0x03 RequestRoutineResults 0x03 执行结果	N	N	N	N	N	N		
0x19	ReadDTCInformation 读取DTC信息	√	√	x	0x01 reportNumberOfDTCByStatusMask 0x01 报告符合MASK状态位DTC的数量	Y	N	Y	Y	Y	Y	13(最小长度) 12(子服务不支持) 13(长度不匹配)	
					0x02 ReportDTCByStatusMask 0x02 通过状态屏蔽报告DTC	Y	N	Y	Y	Y	Y		
					0x04 ReportDTCSnapshotRecordByDTCNumber 0x04 通过DTC报告Snapshot的记录	Y	N	Y	Y	Y	Y		
					0x06 ReportDTCExtendedDataRecordByDTCNumber 0x06 通过DTC报告扩展数据记录	Y	N	Y	Y	Y	Y		
					0x0A ReportSupportedDTC 0x0A 报告支持的DTC	Y	N	Y	Y	Y	Y		
0x14	ClearDiagnosticInformation 清除诊断信息	√	√	N/A	FFFFFF - Clear All DTCs	Y	N	Y	Y	Y	Y	13(最小长度) 31(参数错误) 22(条件不满足)	

CAN 诊断时间参数如下:

Application Layer Timing Parameters

应用层会话管理计时器参数

	Symbol 符号	Min 最小值	Max/Timeout 最大值/超时时间	Unit 单位
ECU 电控 单元	P2 _{Server}	N/A	50	ms
	P2* _{Server}	N/A	5000	ms
	S3 _{Server}	N/A	5000	ms

Network Layer Timing Parameters

网络层定时器参数

	Symbol 符号	Timeout 超时时间	Performance Requirement 性能要求	Unit 单位
ECU 电控 单元	N_As/N_Ar	70	——	ms
	N_Bs	150	——	ms
	N_Br	——	< 70	ms
	N-Cs	——	< 150	ms
	N_Cr	150	——	ms

Other parameters

其它参数

	Symbol 符号	Parameter 参数	Value 值	Unit 单位
ECU 电控 单元	BS	Block Size	0	——
	STmin	Minimum Separation Time	10	ms
	Fill bytes(发送数据填充)	Padding	0xAA	——
	接收填充值检查	ON/OFF	OFF	——
	Fill bytes(接收数据填充)	Padding	——	——
	诊断报文长度	Byte Size	8	——
	诊断报文长度检查	ON/OFF	ON	——
	DCM 接收 BUFFER 最大值	Byte Size	1024	——
	DCM 发送 BUFFER 最大值	Byte Size	1024	——
	TP 半双工			

安全访问算法配置信息如下：

Mask 配置值

Mask = 0x5555AAAAu

密钥算法（根据 Seed 计算 Key）如下，其中 seed 为输入的种子。

安全算法

Key = Seed & Mask

注：最大失败次数为 3，达到最大失败次数启动延时时间为 10S；连续请求种子错误计数不加 1，种子相同，延时时间过后错误计数清零。

DID 列表：

DIDs (HEX)	Name 名称	Diagnostic Session 诊断模式		Security Level 安全级别			Size (Byte)
		(01)	(03)	Locked	Level 1	Level 2	
F186	Active Diag Session 当前诊断会话模式	R	R	R	R	R	1
F121	ECU Serial Number ECU序列号	R	R	R	R	R	16
F190	Vehicle Identification Number 车辆识别编号 (VIN)	R	R/W	R	R/W	R	17
F183	configuration information 配置信息	R	R/W	R	R	R/W	4
F181	applicationSoftwareIdentificationDataIdentifier APP软件编号	R	R	R	R	R	4
0120	DEMO IODID01	R	R/IO	R	R/IO	R	1
0121	DEMO IODID02	R	R/IO	R	R	R/IO	1
0110	DEMO 快照信息 DID01	R	R	R	R	R	1
0111	DEMO 快照信息 DID02	R	R	R	R	R	3

IO DID 列表：

DIDs (HEX)	Name 名称	InputOutput ControlParameter 输入输出 控制参数	Locked	Level 1	Level 2	ControlOption 控制选项	ControlStatus 控制状态	Size (Byte)
0120	DEMO IODID01	00	N	Y	N	—	√	1
		01	N	Y	N	—	√	1
		03	N	Y	N	√	√	1
0121	DEMO IODID02	00	N	N	Y	—	√	1
		01	N	N	Y	—	√	1
		03	N	N	Y	√	√	1

RID 列表：

DIDs (HEX)	Name 名称	RoutineControlType 例程控制 类型	Diagnostic Session 诊断模式		Security Level 安全级别			RoutineControlOption 例程控制 选项	Size (Byte)	Byte	RoutineStatus 例程状态	Size (Byte)
			(01)	(03)	Locked	Level 1	Level 2					
0203	预编程条件检查	01	N	Y	Y	Y	Y	-	0		√	01
0200	DEMO RID01	01	N	Y	N	Y	N	√	1		√	01
0201	DEMO RID02	01	N	Y	N	N	Y	√	1		√	01

DTC 列表：

DTC	DTC Number (HEX)	DTC Description DTC 描述	DTC-Set Condition DTC 设置条件	Faults-Recover Condition 故障恢复条件	DTC Enable Condition DTC 使能条件	DTC Priority DTC 优先级	DTC Displacement DTC 替换	W/Lamp 故障灯状态	Possible Fault Causes 可能故障原因	Corrective Action 维修建议	老化周期	Comment 备注
U018787	C18787	DEMO 通信超时	No Message received in 5 periods 5个帧周期未收到报文（报文0x202）	correct Information Received 报文正常接收	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	1	Y	No 无故障灯	1. CAN connect line failure CAN 连线故障 2. controller failure 控制器故障 3. 具体控制器故障	1. check connect line 检查连线 2. change RM controller 更换控制器 3. 更换具体控制	40	
U007388	C07388	CAN Bus off CAN 总线 Bus off	CAN Bus off 出现 10 次 CAN 总线控制器出现 Busoff 出现 10 次	CAN not in busoff state CAN 从 busoff 状态恢复	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	1	Y	No 无故障灯	1. CAN connect line failure CAN 连线故障 2. controller failure 控制器故障	1. check connect line 检查连线 2. change RM controller 更换控制器	40	
U010001	C10001	SWC 故障 #01	Prefailed 连续 32 次	Prepassed 连续 32 次	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	1	Y	No 无故障灯	NA	NA	NA	不允许老化
U010002	C10002	SWC 故障 #02	Prefailed 连续 32 次	Prepassed 连续 32 次	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	1	Y	No 无故障灯	NA	NA	NA	不允许老化
U010003	C10003	SWC 故障 #03	Prefailed 连续 32 次	Prepassed 连续 32 次	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	1	Y	No 无故障灯	NA	NA	NA	不允许老化
U010004	C10004	SWC 故障 #04	Prefailed 连续 2000ms	Prepassed 连续 2000ms	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	2	Y	No 无故障灯	NA	NA	40	
U010005	C10005	SWC 故障 #05	Prefailed 连续 2000ms	Prepassed 连续 2000ms	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	2	Y	No 无故障灯	NA	NA	40	
U010006	C10006	SWC 故障 #06	Prefailed 连续 2000ms	Prepassed 连续 2000ms	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	2	Y	No 无故障灯	NA	NA	40	
U010007	C10007	SWC 故障 #07	Prefailed 连续 2000ms	Prepassed 连续 2000ms	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	2	Y	No 无故障灯	NA	NA	40	
U010008	C10008	SWC 故障 #08	Prefailed 连续 2000ms	Prepassed 连续 2000ms	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	2	Y	No 无故障灯	NA	NA	40	
U010009	C10009	SWC 故障 #09	Prefailed 连续 20 次	Prepassed 连续 3 次	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	3	Y	No 无故障灯	NA	NA	40	
U01000A	C1000A	SWC 故障 #10	Prefailed 连续 20 次	Prepassed 连续 3 次	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	3	Y	No 无故障灯	NA	NA	40	
U01000B	C1000B	SWC 故障 #11	Prefailed 连续 20 次	Prepassed 连续 3 次	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	3	Y	No 无故障灯	NA	NA	40	
U01000C	C1000C	SWC 故障 #12	Prefailed 连续 20 次	Prepassed 连续 3 次	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	3	Y	No 无故障灯	NA	NA	40	
U01000D	C1000D	SWC 故障 #13	Prefailed 连续 3 次	Prepassed 连续 1 次	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	4	Y	No 无故障灯	NA	NA	40	
U01000E	C1000E	SWC 故障 #14	Prefailed 连续 3 次	Prepassed 连续 1 次	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	4	Y	No 无故障灯	NA	NA	40	
U01000F	C1000F	SWC 故障 #15	Prefailed 连续 3 次	Prepassed 连续 1 次	1. Car Mode 不等于 Drive Mode 2. Voltage 大于 9V 小于 16V 3. 上电后 3000ms 后诊断使能	4	Y	No 无故障灯	NA	NA	40	

注：DTC 格式：01，ISO 14229；DTC status 支持的 bit 位仅 bit7 不支持，0x7F；

DTC 扩展数据:

Record Number	Byte #	Name 名称	Size (Byte)	Update	Byte	Bit	Sub Data Name 子数据名称	Factor 精度	Offset	Min(Phy.) 物理值最小 值	Max(Phy.) 物理值最大 值
1	1	Failed Cycles	1	Y				1	0	0	255
1	1	Occurrence counter	1	Y				1	0	0	255
3	1	Cycles since first failed	1	Y				1	0	0	255
4	1	Cycles since last failed	1	Y				1	0	0	255
5	1	Aging Counter(Up Count)	1	Y				1	0	0	255
6	1	Max FDC during current cycle	1	Y				1	0	0	255
7	1	Max FDC since last clear	1	Y				1	0	0	255
8	1	Current FDC	1	Y				1	0	0	255

DTC 快照:

Record 0x01	Name 名称	Parameter DID 参数ID号	Size (Byte)	Update	Byte	Bit	Sub Data Name 子数据名称	Factor 精度	Offset	Min(Phy.) 物理值最小 值	Max(Phy.) 物理值最大 值
	DEMO 快照信息 DID01	0x0110	1	Y							
	DEMO 快照信息 DID02	0x0111	3	Y							
Record 0x02	Name 名称	Parameter DID 参数ID号	Size (Byte)	Update	Byte	Bit	Sub Data Name 子数据名称	Factor 精度	Offset	Min(Phy.) 物理值最小 值	Max(Phy.) 物理值最大 值
	DEMO 快照信息 DID01	0x0110	1	N							
	DEMO 快照信息 DID02	0x0111	3	N							

3.4 存储协议栈

3.4.1 存储协议栈概述

存储协议栈涉及到的软件模块主要有 Flash、FEE、NvM 模块，其中各个模块的主要功能如下表：

NvM 各配置模块介绍

模块名	功能
Flash	提供对 Flash 的读，写，擦相关操作服务。
FEE	为上层提供虚拟线性地址空间和统一的存储分配方案。
NvM	非易失性数据的存储和管理。

3.4.2 存储协议栈配置说明

存储协议栈中主要配置了如下 NvMBlock:

NvMBlock 名	作用
NvMBlock_ConfigID	NvM 管理
NvMBlock_Dem_Data	用来存放 Dem 的数据
NvMBlock_Dem_Status	用来存放 Dem 的状态
NvMBlock_Dcm	用来存放 Dcm 的数据（暂未使用）
NvMBlock_SecurityLevel01	用来存放安全等级 1 错误计数（暂未使用）
NvMBlock_SecurityLevel02	用来存放安全等级 2 错误计数（暂未使用）
NvMBlock_Did_0xF190	用来存放 DID 0xF190 的数据
NvMBlock_Did_0xF183	用来存放 DID 0xF183 的数据

3.5 看门狗协议栈

3.5.1 看门狗协议栈概述

看门狗协议栈是一种用于监控和保护系统运行状态的机制。它通过监控软件执行的稳定性和正确性确保了在系统发生故障时能迅速采取恢复措施。

Wdg 协议栈主要涉及到的模块为 Wdg、WdgIf、WdgM，其中各个模块的主要功能如下表：

Wdg 看门狗协议栈各配置模块介绍

模块名	功能
Wdg	Wdg 属于 MCAL 的一部分，用于完成看门狗初始化，模式设置以及喂狗设置等。
WdgIf	WdgIf 模块属于 ECU 抽象层，能够允许上层 WdgM 模块来同时处理多个看门狗实体，比如外部看门狗或者内部看门狗。
WdgM	WdgM 模块从硬件看门狗实体监控的过程抽象出来完成软件程序执行监控抽象。

3.5.2 看门狗协议栈配置说明

看门狗协议栈中配置了一个 Alive supervision 用于监控定期软件的时间。

Alive 监控参数配置

监控类型	描述	监控实体个数	监控点个数	参考周期	监控失败门限	监控失效门限	期望执行次数	次数上偏差	次数下偏差
Alive 监控	监控一次 mainfunction 周期alive监控点执行的次数	1	1	1	0	0	1	0	0

3.6 OS 协议栈

3.6.1 OS 协议栈概述

AUTOSAR OS 主要负责任务管理和中断管理功能；实现包括以下模块:Task、Isr、Countor、Alarm、ScheduleTable、Event、Resource 等。

3.6.2 OS 协议栈配置说明

OsTask 配置

Name	Priority	Stack Size[4Bytes]	Preemptive Poilcy	OsTaskAutostart	Core
iSoft_Auto_DEFAULT_OsTask_Init	1	512	NON	True	0
iSoft_Auto_OsTask_1ms_BSW	4	512	FULL	False	0
iSoft_Auto_OsTask_5ms_BSW	3	512	FULL	False	0
iSoft_Auto_OsTask_10ms_BSW	2	512	FULL	False	0
iSoft_Auto_OsTask_100ms	1	512	FULL	False	0
OsTask_100ms_c1	1	256	FULL	False	1
OsTask_100ms_c2	1	256	FULL	False	2
OsTask_100ms_c3	1	256	FULL	False	3

OsIsr 配置

Name	Category	Stack Size[4Bytes]	Priority	Nested Enable	Core
RCANGERR0	GATEGORY_2	512	1	False	0
RCANGRECC0	GATEGORY_2	512	2	False	0
RCAN2ERR	GATEGORY_2	512	3	False	0
RCAN2REC	GATEGORY_2	512	4	False	0
RCAN2TRX	GATEGORY_2	512	5	False	0
WDTB0TIT	GATEGORY_2	512	6	False	0

OsAlarm 配置

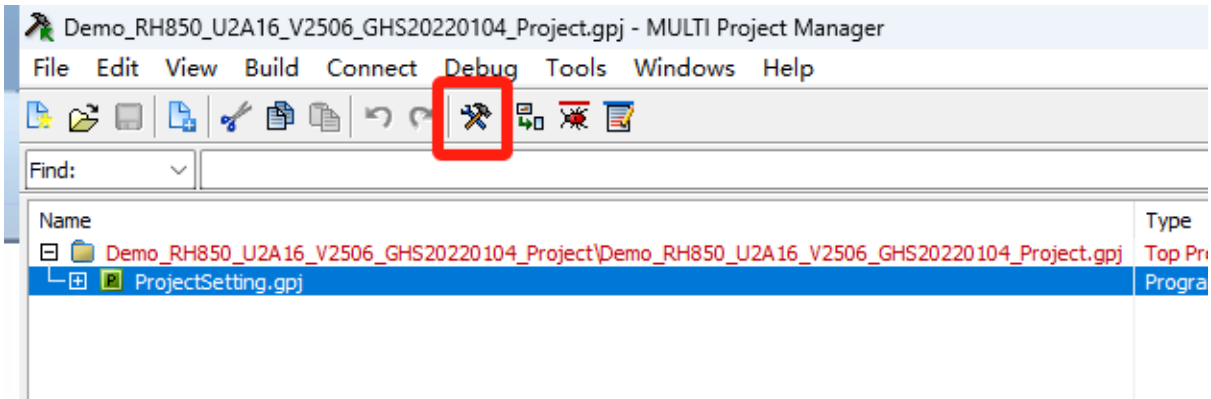
Name	Set Event	Set Event Task	Core
iSoft_Auto_OsAlarm_1ms_BSW	iSoft_Auto_OsTask_1ms_BSW	iSoft_Auto_OsTask_1ms_BSW	0

iSoft_Auto_OsAlarm_5ms_BSW	iSoft_Auto_OsTask_5ms_BSW	iSoft_Auto_OsTask_5ms_BSW			0
iSoft_Auto_OsAlarm_10ms_BSW	iSoft_Auto_OsTask_10ms_BSW	iSoft_Auto_OsTask_10ms_BSW			0
Name	Activate Task	OsAlarmAutostart	Start Time	Cycle Time	Core
iSoft_Auto_OsAlarm_100ms	iSoft_Auto_OsTask_100ms	True	100	100	0
OsAlarm_100ms_c1	OsTask_100ms_c1	True	100	100	1
OsAlarm_100ms_c2	OsTask_100ms_c2	True	100	100	2
OsAlarm_100ms_c3	OsTask_100ms_c3	True	100	100	3

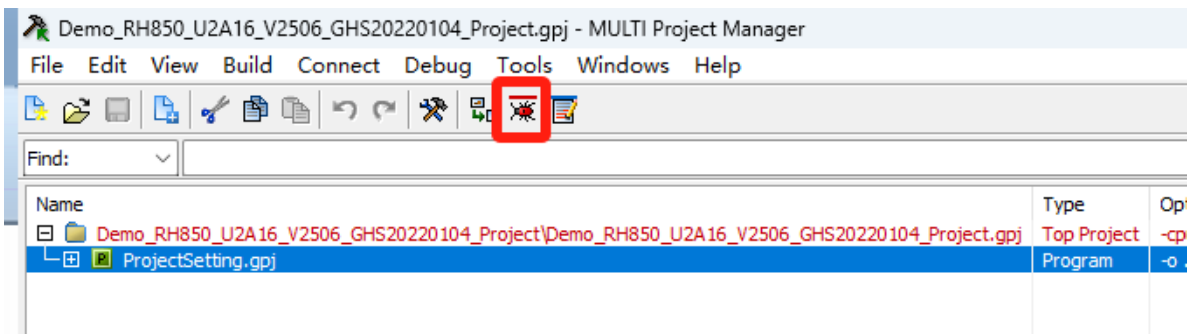
4. 工程验证

4.1 编译及下载

将 BSW 工程导入 IDE 后点击 **Build Project** 即可编译工程。



编译通过后点击 **Debug** 下载工程。（根据使用的调试器配置）

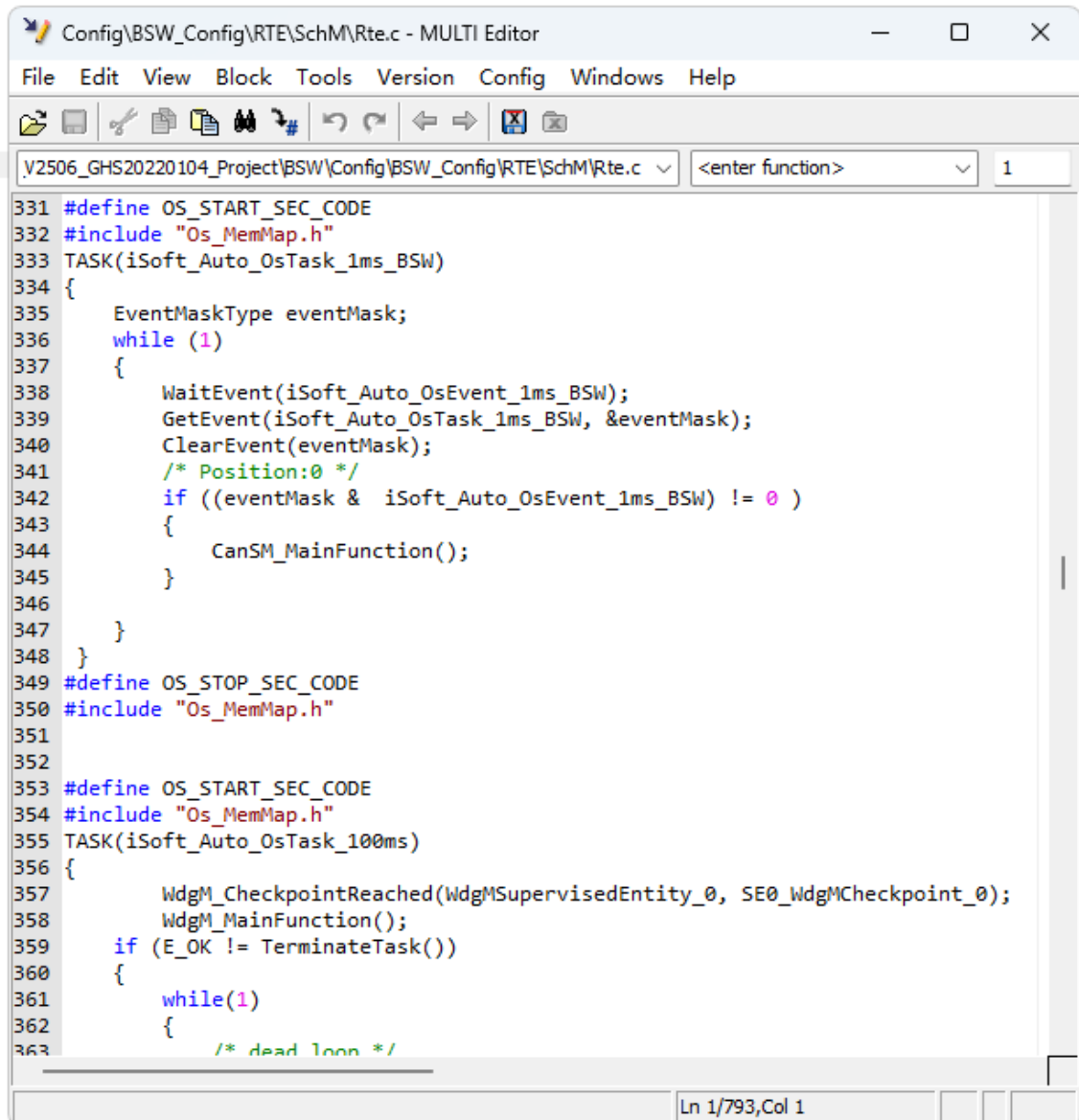


4.2 OS 任务

工程中执行的 OS 任务在

Demo_RH850_U2A16_V2504_GHS20220104_Project

\BSW\Config\BSW_Config\RTE\SchM 路径下的 Rte.c 文件中，用户可以自行在相关的任务中添加自己需要实现的功能。



The screenshot shows a MULTI Editor window titled 'Config\BSW_Config\RTE\SchM\Rte.c - MULTI Editor'. The window contains C code for two tasks. The first task, 'iSoft_Auto_OsTask_1ms_BSW', is defined with a while loop that calls 'WaitEvent', 'GetEvent', 'ClearEvent', and 'CanSM_MainFunction()'. The second task, 'iSoft_Auto_OsTask_100ms', is defined with a while loop that calls 'WdgM_CheckpointReached', 'WdgM_MainFunction()', and 'TerminateTask()'. The code is color-coded, and the status bar at the bottom indicates 'Ln 1/793, Col 1'.

```
331 #define OS_START_SEC_CODE
332 #include "Os_MemMap.h"
333 TASK(iSoft_Auto_OsTask_1ms_BSW)
334 {
335     EventMaskType eventMask;
336     while (1)
337     {
338         WaitEvent(iSoft_Auto_OsEvent_1ms_BSW);
339         GetEvent(iSoft_Auto_OsTask_1ms_BSW, &eventMask);
340         ClearEvent(eventMask);
341         /* Position:0 */
342         if ((eventMask & iSoft_Auto_OsEvent_1ms_BSW) != 0 )
343         {
344             CanSM_MainFunction();
345         }
346     }
347 }
348 }
349 #define OS_STOP_SEC_CODE
350 #include "Os_MemMap.h"
351
352
353 #define OS_START_SEC_CODE
354 #include "Os_MemMap.h"
355 TASK(iSoft_Auto_OsTask_100ms)
356 {
357     WdgM_CheckpointReached(WdgMSupervisedEntity_0, SE0_WdgMCheckpoint_0);
358     WdgM_MainFunction();
359     if (E_OK != TerminateTask())
360     {
361         while(1)
362         {
363             /* dead loop */
```

4.3 工程运行验证

正确连接开发板并成功下载运行后，使用通信测试验证工具接收板子发出的报文，可以收到如下报文。

序号	时间标识	源通道	帧ID	周期(s)	帧数量	CAN类型	方向	长度	数据
0	40.178400	0	0x501	0.819967	50	CAN	Rx	8	01 50 00 00 00 00 00 00
1	40.598400	0	0x300	0.099995	407	CAN	Rx	8	00 00 00 00 00 00 00 00
2	40.518400	0	0x302	0.499977	82	CAN	Rx	8	00 00 00 00 00 00 00 00
3	40.623400	0	0x303	0.049998	813	CAN	Rx	8	00 0C 00 00 00 00 00 00
4	40.628400	0	0x360	0.099996	407	CAN	Rx	8	00 00 00 00 00 00 00 00